

第20回 JavaScript の基本 5

実習時のフォルダ構成

第 19 回と同じ形式で、`Web_名前` の中に `20` フォルダを作る。`20` の中に `sample` と `exercise` を作り、次のファイルを用意する。

```
Web_名前
├── 20
│   ├── sample
│   │   ├── index.html
│   │   └── main.js
│   └── exercise
│       ├── index.html
│       └── main.js
```

- `sample`：講義で使うサンプル（講義内容のコードは、すべて `sample/main.js` に書く）
- `exercise`：演習で作るファイル（演習 1～2、発展）

`sample/index.html` には、次の内容を書く。今回は、得点を表示する部分とボタンを HTML に用意しておく。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>JavaScriptの基本 5</title>
</head>
<body>
  <p>SCORE: <span id="score">0</span></p>
  <button id="scoreButton">得点を増やす</button>
  <script src="main.js"></script>
</body>
</html>
```

- `SCORE:` の後ろの数字だけを `<span>` で囲み、`id="score"` を付けている。あとで JavaScript から、この数字の部分だけを書き換える。
- `<button>` はボタンを表示するタグ。`id="scoreButton"` を付けている。

`exercise/index.html` は、`sample/index.html` をコピーして、`<p>` と `<button>` の 2 行を消した形から始める（演習 2 で書き足す）。

講義内容

以降の講義内容のコードは、`sample/main.js` に、上から順に書き足していく。書いたら

`sample/index.html` をブラウザで開き、`F12` キーの開発者ツール（コンソール）と、画面の表示で結果を確認する。

1. 前回の復習

- `for` 文で、同じ処理を繰り返せる。`配列[i]` と組み合わせると、配列のすべての要素を順に処理できる。

```
const scores = [80, 45, 92];
for (let i = 0; i < scores.length; i++) {
  console.log(scores[i]);      // 80 45 92
}
```

- 今回は、処理にまとめて名前を付ける「関数」と、JavaScript で Web ページの表示を変える方法を学ぶ。

2. 関数と引数

いくつかの処理をまとめて名前を付けたものを「関数」という。一度作っておけば、名前を書くだけで何回でも実行できる。

```
function 関数名() {
  実行する処理
}
```

```
function showTitle() {
  console.log("=== SHOOTING GAME ===");
}

showTitle();
showTitle();
```

- `function showTitle() { }` の部分は、関数を**作る**（定義する）だけ。この時点では、中の処理は実行されない。
- `showTitle();` のように、名前の後ろに `( )` を付けて書くと、関数の中の処理が実行される（関数を**呼び出す**）。
- 今までに使ってきた `console.log( )` や `Math.random( )` も、JavaScript に最初から用意されている関数の仲間である。
- この授業では、読みやすくするため、関数は、呼び出すより前（ファイルの上の方）に作ることを基本にする。

関数に値を渡して、その値を使った処理をさせることもできる。渡す値を「引数（ひきすう）」という。

```
function showDamage(damage) {
  console.log(` ${damage} のダメージ! `);
}

showDamage(10);      // 10のダメージ!
showDamage(25);      // 25のダメージ!
```

- `showDamage(10);` と呼び出すと、`( )` の中の `10` が、関数の `damage` に入ってから、中の処理が実行される。

- 引数は、`,` で区切って、いくつでも渡せる（例：`function showStatus(name, hp)`）。渡す順番と、受け取る順番は同じになる。

3. 戻り値

関数の中で計算した結果を、呼び出した場所へ返すことができる。返す値を「戻り値」といい、`return` で返す。

```
function calcDamage(attack, defense) {  
  return attack - defense;  
}  
const damage = calcDamage(40, 15);  
console.log(damage);           // 25
```

- `calcDamage(40, 15)` の部分が、戻り値の `25` に置き換わると考える。
- `return` が実行されると、関数はそこで終わる。

4. 関数の外の変数を、関数の中で使う・書き換える

関数の外（ファイルの上の方）で作った変数は、関数の中でも使え、書き換えることもできる。

```
let score = 0;  
function addScore(point) {  
  score += point;  
}  
addScore(100);  
addScore(50);  
console.log(score);           // 150
```

- 得点のように、ゲーム全体で使い続ける値は、関数の外で作る。
- 関数の中で作った変数（引数も含む）は、その関数の中だけで使える。関数の外からは使えない。

5. HTML・CSS・JavaScript の役割と DOM

ここまでは、結果をコンソールに表示してきた。ここからは、JavaScript で Web ページの表示そのものを変える。

技術	役割
HTML	ページの部品（文字・ボタンなど）を用意する
CSS	部品の見た目を整える
JavaScript	部品を取り出して、中身を書き換えたり、クリックに反応したりする

ブラウザは、HTML のタグを 1 つずつ「部品」として覚えている。JavaScript は、この部品を取り出して、中身を書き換えられる。この仕組みを **DOM** という。

```
index.html の <span id="score">0</span>
  ↓ document.getElementById("score") で取り出す
JavaScript の scoreElement
  ↓ scoreElement.textContent = score; で書き換える
画面の表示が「SCORE: 150」に変わる
```

6. 要素を取り出して、表示を書き換える

```
const scoreElement = document.getElementById("score");
scoreElement.textContent = score; // 画面の「0」が「150」に変わる
```

- `document.getElementById("score")` : HTMLの中から、`id` が `score` の要素（ここでは `<span>`）を取り出す。`document` は、ブラウザに表示している HTML のページ全体のこと。
- 取り出した要素は、変数（ここでは `scoreElement`）に入れて使う。
- `要素.textContent = 値;` : 要素の中の文字を、その値に書き換える。数値を入れると、文字として表示される。
- `<script>` を `</body>` の直前に書いているので、JavaScript が動くときには、HTML の部品がもう読み込まれている。そのため、`getElementById` で取り出せる（第 16 回の「この授業では `</body>` の直前に書く」の理由の 1 つ）。
- `getElementById` の `"score"` と、HTML の `id="score"` は、同じ名前にする。違うと、コンソールに `Cannot set properties of null` というエラーが出る。

7. ボタンのクリックで関数を呼び出す

```
const scoreButton = document.getElementById("scoreButton");

function addTenPoints() {
  addScore(10);
  scoreElement.textContent = score;
}

scoreButton.addEventListener("click", addTenPoints);
```

- `要素.addEventListener("click", 関数名);` : その要素がクリックされたら、指定した関数を呼び出すように、ブラウザに頼む。
- 関数名の後ろには `()` を付けない（`addTenPoints()` とは書かない）。`()` を付けると、クリックを待たずに、その場で呼び出してしまう。ここでは、関数そのものをブラウザに渡している。
- この授業では、まず、クリックで呼び出す関数を、引数なしで作る方法を使う。そのため、講義 4 の `addScore(point)` を直接渡さず、引数なしの `addTenPoints` を作り、その中から `addScore(10)` を呼び出している。
- ボタンを押すたびに、得点が 10 増えて、画面の表示が変わることを確認する。
- この「表示する部品を HTML に用意して、JavaScript で中身を書き換える」形で、最後に作るシューティングゲームの SCORE（得点）や LIVES（残機）を表示する。ボタンのクリックは、ゲームのリスタートボタンで使う。

演習

`exercise/main.js` に書く。結果は、`exercise/index.html` をブラウザで開いて、コンソールと画面で確認する。

演習 1 関数を作って呼び出す

- 「GAME OVER」と表示する関数 `showGameOver` を作り、2 回呼び出す。
- 得点を引数で受け取り、「得点：〇〇点」と表示する関数 `showScore` を作って、呼び出す。

演習 2 ボタンで残機を減らす

- `exercise/index.html` の `<script>` の上に、次の 2 行を書き足す。

```
<p>LIVES: <span id="lives">3</span></p>
<button id="damageButton">残機を減らす</button>
```

- `exercise/main.js` に、次を書く。
- 残機を入れる変数を `let` で作る（最初は 3）。
- `getElementById` で、`id` が `lives` の要素と、`damageButton` の要素を取り出して、変数に入れる。
- 残機を 1 減らし、`textContent` で表示を書き換える関数 `loseLife` を、引数なしで作る。
- `addEventListener` で、ボタンがクリックされたら `loseLife` が呼び出されるようにする。
- ボタンを押すたびに、画面の LIVES の数字が 1 ずつ減ることを確認する。

発展（任意）

- 倒した敵の数を引数で受け取り、「敵の数 × 100」を戻り値として返す関数 `calcScore` を作る。5 を渡して、戻り値を表示する。
- HTML に `<p id="message"></p>` を書き足す。残機が 0 以下になったら、この要素に「GAME OVER」と表示する。
- HTML に「リセット」ボタン（`id="resetButton"`）を書き足す。押すと、残機が 3 に戻り、「GAME OVER」の表示が消える（空の文字 `""` を入れる）ようにする。